TECHNICAL DOCUMENTATION

v2.0.0

Umpire AI

Cricket ball trajectory analysis system with hybrid deep-learning and classical computer vision pipeline. Complete project documentation covering architecture, deployment, and API reference.

ARCHITECTURE
Python FastAPI + React 18

DETECTION
YOLOv11 + HSV Hybrid

TRAJECTORY
Unscented Kalman Filter

LICENSE
MIT

Table of Contents

1. Project Overview

Umpire AI is a cricket decision-support system designed to assist umpires in making accurate dismissal decisions by analyzing video footage from the umpire camera view. The system uses a hybrid deep-learning and classical computer vision pipeline to detect the cricket ball in video frames, track it across frames, estimate its full trajectory using physics-based modeling, and then apply ICC rules to determine whether a batsman should be given out. The project is split into two independent, deployable components: a Python backend for video processing and a React frontend for interactive analysis.

1.1 Problem Statement

In professional cricket, critical dismissal decisions such as LBW (Leg Before Wicket) require determining whether the ball would have hit the stumps had the batsman's pad not intercepted it. Currently, this is done by the on-field umpire's judgment, with the Decision Review System (DRS) using Hawk-Eye technology that requires 6-8 specialized cameras at 300 FPS costing millions of dollars. Umpire AI aims to provide similar decision support using only a single standard camera view (the umpire's perspective), making it accessible for domestic, club, and training-level cricket where Hawk-Eye is not available.

1.2 Solution Architecture

The system accepts cricket match video uploads through a web interface, processes them through a multi-stage computer vision pipeline on the backend, and returns annotated results including the ball trajectory overlay, a 2D pitch map showing the ball's path, and a final decision with confidence score and reasoning. The v2 hybrid pipeline combines YOLOv11 for ball detection with an Unscented Kalman Filter for trajectory estimation, automatically falling back to classical methods (HSV color segmentation, Hungarian algorithm tracking, linear Kalman filtering) when GPU resources or model files are not available.

Metric	Value
Detection Methods	YOLOv11 + P2 Head + HSV Fallback
Tracking Methods	BoT-SORT + Hungarian Algorithm Fallback
Trajectory Model	UKF with Gravity, Drag, Magnus, Bounce
Decision Rules	ICC Standard: LBW, Bowled, Wide, No-ball
Max Video Size	500 MB
Pitch Dimensions	20.12m x 3.05m (ICC Standard)
State Vector	6D: [x, y, z, vx, vy, vz]

Prediction Steps	20 future trajectory points

Table 1. Umpire AI v2 Key Technical Specifications

2. System Architecture

Umpire AI follows a client-server architecture with complete separation between frontend and backend. The frontend is a single-page application (SPA) built with React 18 and TypeScript, communicating with the backend exclusively through a REST API. The backend is a FastAPI application that orchestrates the computer vision pipeline, manages video storage, and provides analysis results as JSON responses. This separation allows each component to be developed, deployed, and scaled independently.

2.1 Processing Pipeline

When a video is uploaded and analysis is requested, the backend executes a six-stage pipeline. First, the video is split into individual frames. Second, the ball detection stage identifies the ball in each frame using YOLOv11 (GPU) or HSV color segmentation with MOG2 background subtraction (CPU fallback). Third, the ball tracker links detections across frames using BoT-SORT with Camera Motion Compensation and Noise Information Criterion, falling back to the Hungarian algorithm for optimal global assignment. Fourth, the trajectory estimator uses an Unscented Kalman Filter with a full physics model including gravity, quadratic air drag, Magnus effect for lateral swing, and a bounce model with configurable coefficient of restitution. Fifth, the pitch mapper performs hierarchical line clustering on detected Canny+Hough lines to identify crease pairs, then applies a perspective transform to map pixel coordinates to standard pitch dimensions. Finally, the decision engine evaluates ICC rules in priority order to produce the umpire decision with detailed reasoning.

Stage	Component	Input	Output
1. Detection	BallDetector	Video frames	Ball bounding boxes
2. Tracking	BallTracker	Detections per frame	Linked track IDs
3. Trajectory	TrajectoryEstimator	Ball positions	6D state + predicted path
4. Pitch Mapping	PitchMapper	Video frame	Perspective transform matrix
5. Decision	DecisionEngine	Trajectory + pitch data	OUT/NOT OUT + reason
6. Annotation	VideoProcessor	All results	Annotated video + JSON

Table 2. Six-Stage Video Processing Pipeline

2.2 Backend Architecture

The backend is structured as a modular Python package following clean architecture principles. The entry point is **app/main.py**, which creates the FastAPI application, configures CORS middleware, and includes API route modules. The **app/core/** directory contains the four core computer vision modules (ball_detector, ball_tracker, trajectory_estimator, pitch_mapper) plus the decision engine. The **app/services/** directory contains the video_processor.py which orchestrates the pipeline. API routes in **app/api/routes/** handle video upload and analysis requests. Configuration is managed through **app/config.py** using Pydantic Settings, loading from environment variables and .env files. All data schemas are defined in **app/models/schemas.py** using Pydantic BaseModel classes for request/response serialization.

2.3 Frontend Architecture

The frontend is a React 18 SPA built with Vite 5 for fast development and optimized production builds. The application uses React Router DOM v6 for client-side routing with three main pages: UploadPage (video upload with drag-and-drop), ProcessingPage (6-step animated pipeline visualization), and AnalysisPage (frame-by-frame player with trajectory overlay, pitch map, and decision panel). The **src/hooks/** directory contains custom React hooks for video upload management, frame-by-frame playback control, and analysis data fetching. The **src/lib/api.ts** module provides a centralized API client for all backend communication. All styling uses Tailwind CSS 3 with a dark sports-tech theme using emerald, amber, and red accent colors. HTML5 Canvas is used for the trajectory overlay and 2D pitch map rendering.

3. Core Modules - Technical Deep Dive

3.1 Ball Detection (Hybrid YOLOv11 + HSV)

Ball detection is the most critical stage of the pipeline. The v2 system uses a hybrid approach that automatically selects the best available method. When a GPU and trained model are available, it uses YOLOv11s with a P2 (320x320) detection head specifically optimized for small objects as tiny as 3-5 pixels. The P2 head adds a high-resolution feature map layer to the standard YOLO architecture, which is essential for detecting cricket balls that appear very small in broadcast footage (often under 10 pixels in diameter). The model runs at approximately 100 FPS on an NVIDIA T4 GPU, making it suitable for real-time applications. The single-class training dedicates all model capacity to ball detection, improving accuracy over multi-class models.

When no GPU or model file is available, the system falls back to classical HSV color segmentation combined with MOG2 (Mixture of Gaussians) background subtraction. This approach converts each frame to HSV color space, applies configurable color range masks for red, white, or pink balls, and uses morphological operations (erosion and dilation) to remove noise. The MOG2 background subtractor helps identify moving objects, reducing false positives from static similarly-colored objects like clothing

or stadium elements. The detection tier is configurable via the DETECTION_TIER environment variable (auto/yolo/classical).

3.2 Ball Tracking (Hybrid BoT-SORT + Hungarian)

The tracking module links ball detections across consecutive frames to form a continuous trajectory. The GPU path uses BoT-SORT (Bootstrapping BoT-SORT), which provides two key capabilities beyond simple tracking. First, Camera Motion Compensation (CMC) handles broadcast camera pan, tilt, and zoom movements that would otherwise cause false detections or track breaks. CMC estimates the camera's affine transformation between frames using feature matching and compensates for it before data association. Second, the Noise Information Criterion (NIC) down-weights detections from blurry or low-quality frames, preventing degraded detections from corrupting the track state. The tracker is configured with a 30-frame track buffer to handle occlusions where the ball disappears behind the bowler's body or bat.

The CPU fallback uses the Hungarian algorithm (scipy.optimize.linear_sum_assignment) for globally optimal detection-to-track assignment. Unlike the v1 greedy nearest-neighbor approach, which simply matched each detection to the closest existing track, the Hungarian algorithm solves the assignment problem optimally across all detections simultaneously, producing better results when multiple objects are close together or when detections are noisy. The tracking tier is configurable via TRACKING_TIER (auto/botsort/classical).

3.3 Trajectory Estimation (UKF with Physics Model)

The trajectory estimator is the scientific core of Umpire AI. The v2 system uses an Unscented Kalman Filter (UKF) with a 6-dimensional state vector [x, y, z, vx, vy, vz] representing the ball's 3D position and velocity. Unlike the v1 linear Kalman filter which assumed constant velocity, the UKF uses a full physics-based process model that includes four forces acting on the ball during flight.

Force	Formula Component	Purpose
Gravity	F = -mg (downward)	Constant gravitational acceleration (9.81 m/s2)
Air Drag	F = -Cd * v2 (opposing velocity)	Quadratic air resistance proportional to speed squared
Magnus Effect	F = spin x velocity (lateral)	Spin-induced lateral force for swing/seam movement
Bounce	vz = -e * vz (reversal)	Coefficient of restitution model for pitch bounce

Table 3. Physics Model Forces in the UKF Process Model

The UKF is preferred over the Extended Kalman Filter (EKF) because it handles the nonlinear dynamics of ball bouncing without requiring Jacobian computation. The sigma-point transform naturally propagates the nonlinear state transition through the physics model, producing accurate mean and

covariance estimates even when the ball undergoes sudden velocity changes during bounces. The UKF also provides a 20-step future trajectory prediction capability, which is critical for LBW decisions where the system must determine if the ball would have hit the stumps. All physics parameters (gravity, drag coefficient, Magnus coefficient, restitution) are configurable via environment variables. The linear KF fallback uses a constant-velocity 2D model [x, y, vx, vy] with linear regression for stump-hit prediction.

3.4 Pitch Mapping

The pitch mapper transforms pixel coordinates from the video frame to standard pitch coordinates using a perspective transform. The v2 system uses hierarchical DBSCAN clustering to group detected Canny+Hough lines by angle and proximity, identifying bowling crease and batting crease pairs. Sub-pixel Canny edge detection thresholds are computed from image statistics using Otsu's method, adapting to varying lighting conditions. Line merging produces robust representative crease lines from clustered groups. The perspective transform maps the four detected pitch corners to a 20.12m x 3.05m standard pitch rectangle following ICC specifications. If automatic calibration fails, the system supports manual calibration through the API by providing four corner points.

3.5 Decision Engine

The decision engine evaluates ICC cricket rules in strict priority order to produce the umpire decision. The evaluation order is: No-ball (was the delivery legal?), Wide (did the ball pass outside the batsman's reach?), Bowled (did the ball hit the stumps directly?), and LBW (Leg Before Wicket - the most complex decision requiring three simultaneous conditions). For LBW, the engine checks: (1) Pitch in Line - did the ball pitch in line with the stumps or on the off side?, (2) Impact in Line - did the ball hit the pad in line with the stumps?, and (3) Would Hit Stumps - would the ball have continued on to hit the stumps? All three conditions must be met for an LBW OUT decision. Each decision includes a confidence score (0-100%) and a human-readable reason string explaining the ruling.

4. API Reference

The backend exposes a REST API built with FastAPI. All endpoints are prefixed with /api and return JSON responses with proper HTTP status codes. The API supports asynchronous processing: video upload returns immediately with a video_id, analysis is started as a background task returning a job_id, and progress can be polled via the status endpoint or streamed via Server-Sent Events (SSE). Interactive API documentation is available at the /docs (Swagger UI) and /redoc (ReDoc) endpoints.

Method	Endpoint	Description	Status Codes

POST	/api/upload	Upload cricket video file	200, 400
GET	/api/videos	List all uploaded videos	200
GET	/api/videos/{id}	Get video metadata	200, 404
DELETE	/api/videos/{id}	Delete uploaded video	200, 404
POST	/api/analyze/{id}	Start analysis (returns job_id)	200, 404
GET	/api/analysis/{id}/status	Poll job progress (0-100%)	200
GET	/api/analysis/{id}/result	Full analysis + decision	200, 202, 500
GET	/api/analysis/{id}/frames	Frame-by-frame data (paginated)	200, 202
GET	/api/analysis/{id}/video	Download annotated video	200, 202, 404
GET	/api/analysis/{id}/stream	SSE progress stream	200
GET	/health	Backend health check	200

Table 4. Complete Backend API Endpoint Reference

4.1 Key Response Schemas

The AnalysisResultResponse is the primary response containing the complete analysis output. It includes the decision (DecisionResponse with decision, dismissal_type, confidence, reason, ball_speed_kmh), the full trajectory as a list of TrajectoryPointSchema objects (x, y, z, frame_number, timestamp), summary statistics (total_frames, frames_with_ball, deviation_degrees, predicted_stump_hit), and processing metadata (processing_time, annotated_video_path). Frame-by-frame data is available separately through the frames endpoint, which supports pagination with offset and limit parameters. Each FrameDataSchema includes the frame number, timestamp, ball_detected flag, and optional BallPositionSchema with pixel coordinates, radius, and confidence score.

5. Frontend Features

The frontend provides a rich, interactive analysis experience through five main components. The **Video Uploader** supports drag-and-drop file upload with real-time progress tracking, ball type selection (Red/White/Pink), file validation (format and size), and video preview before analysis. The **Frame-by-Frame Player** is built on HTML5 video with precise frame stepping (forward and backward), speed controls (0.25x, 0.5x, 1x, 2x), loop markers for IN/OUT points, keyboard shortcuts (Space for play/pause, Arrow keys for frame stepping), frame counter with timecode display (MM:SS:FF), and a Canvas overlay synchronized with video playback for trajectory visualization.

The **Trajectory Canvas** renders a progressive trajectory visualization that builds as the video plays, with a ball position glow and crosshair, gradient trajectory path (green to amber to red representing delivery progression), stumps drawing at the batting end, and devicePixelRatio-aware rendering for crisp display on high-DPI screens. The **Pitch Map** provides a 2D top-down view with accurate cricket pitch dimensions (20.12m x 3.05m), crease lines, return creases, stumps at both ends, ball trajectory as an amber dashed line, pitch point marker, impact point marker, wicket zone highlight, deviation angle indicator, and batsman position marker. The **Decision Panel** displays OUT/NOT OUT with animated glow, dismissal type badge, confidence percentage with color-coded bar, decision reason explanation, and quick statistics including speed, deviation, predicted path, and processing time.

6. Configuration Reference

All configuration is managed through environment variables loaded from a .env file using Pydantic Settings. The system provides extensive customization options divided into six categories: detection parameters, tracking parameters, trajectory/physics parameters, pitch dimensions, video processing settings, and general application settings. The tier selection system (DETECTION_TIER, TRACKING_TIER) allows choosing between auto (recommended), deep-learning only, or classical-only modes. The auto mode provides the best of both worlds by attempting the advanced method first and falling back gracefully if dependencies are missing.

Variable	Default	Category	Description
DETECTION_TIER	auto	Detection	auto, yolo, or classical
TRACKING_TIER	auto	Tracking	auto, botsort, or classical
USE_UKF	true	Trajectory	Unscented KF (true) or Linear KF (false)
YOLO_MODEL_PATH	yolo11s_cricket_ball.pt	Detection	Path to trained YOLO weights
YOLO_CONFIDENCE	0.35	Detection	Minimum detection confidence threshold
GRAVITY	9.81	Physics	Gravitational acceleration (m/s2)
AIR_DRAG_COEFF	0.002	Physics	Simplified air drag coefficient
MAGNUS_COEFF	0.001	Physics	Spin-induced lateral force coefficient
RESTITUTION	0.65	Physics	Bounce coefficient of restitution
BALL_TYPE	red	General	Ball color: red, white, or pink
FRAME_SKIP	2	Processing	Process every Nth frame for speed
MAX_VIDEO_SIZE	524288000	Upload	Maximum upload size (500 MB)

Table 5. Key Configuration Variables

7. Deployment Guide

Umpire AI is designed for independent deployment of frontend and backend. Each component can be deployed on different platforms, connected via the VITE_API_URL environment variable that tells the frontend where to find the backend API. This section covers the recommended deployment options for each component, along with Docker-based self-hosting for users who prefer full control.

7.1 Backend Deployment Options

Railway (Recommended) - Railway provides the easiest deployment experience with automatic Git push-to-deploy. Create a new project at railway.app, connect your GitHub repository, and Railway will auto-detect Python as the runtime. Set the start command to "uvicorn app.main:app --host 0.0.0.0 --port $PORT" and configure environment variables from the .env.example file. Railway offers a free tier with 500 hours/month and a hobby plan at $5/month with 1GB RAM for faster video processing.

Render.com - Render provides a similar experience with a generous free tier. Create a new Web Service, connect your GitHub repo, and configure the build command (pip install -r requirements.txt), start command (uvicorn app.main:app --host 0.0.0.0 --port $PORT), and environment variables. The free tier spins down after 15 minutes of inactivity, resulting in a ~30 second cold start for the first request after a period of inactivity. Paid plans start at $7/month for consistent performance.

VPS (DigitalOcean, AWS EC2) - For full control, deploy on any VPS with Python 3.11+. Clone the repository, create a virtual environment, install the CPU-only dependencies, copy .env.example to .env, and run the server. A systemd service file is recommended for production to ensure the server starts automatically on boot and restarts on failure.

7.2 Frontend Deployment Options

Vercel (Recommended) - Vercel provides the best experience for React SPAs with automatic Git integration and a global CDN. Import your GitHub repository at vercel.com, set the framework preset to Vite, and configure the VITE_API_URL environment variable to point to your deployed backend URL. Vercel automatically handles SSL certificates, CDN distribution, and continuous deployment from your main branch. The free tier provides 100GB bandwidth per month with unlimited sites.

Netlify - Netlify offers similar functionality to Vercel for static site deployment. Connect your GitHub repository, set the build command to "npm run build" and the publish directory to "dist". Set the VITE_API_URL environment variable to your backend URL. Netlify's free tier includes 100GB bandwidth and automatic HTTPS.

7.3 Docker Deployment (Self-Hosted)

For users who prefer self-hosting, the project includes a production docker-compose.yml that orchestrates three Docker containers: the backend (FastAPI on port 8000), the frontend (nginx serving the React build on port 80), and an nginx reverse proxy on port 80/443 that routes traffic. The reverse proxy routes /api/* requests to the backend and all other requests to the frontend, enabling both components to be accessed from a single domain. Docker volumes are used for persistent storage of uploaded videos and analysis outputs. A health check ensures the backend is fully started before the frontend begins accepting requests.

Platform	Component	Free Tier	Cold Start	Best For
Railway	Backend	500 hrs/month	No	Easy Python deployment
Render	Backend	Yes (spins down)	~30 seconds	Python web services
Vercel	Frontend	100GB bandwidth	No	React SPA, CDN
Netlify	Frontend	100GB bandwidth	No	Static sites, SPA
Docker/VPS	Both	No (server cost)	No	Full control, privacy

Table 6. Deployment Platform Comparison

8. GitHub Repository Setup

The project is organized into two independent GitHub repositories, one for the backend and one for the frontend. This separation allows independent versioning, deployment, and collaboration. Both repositories include comprehensive .gitignore files that exclude virtual environments, model weights (too large for Git), uploaded videos, output files, IDE configurations, and environment/secret files. Each repository also includes a .env.example file documenting all configurable parameters with their defaults and descriptions.

8.1 Creating the Repositories

To create the repositories, navigate to github.com/new and create two new repositories. The backend repository should be named "umpire-ai-backend" with the description "Cricket ball trajectory analysis backend - YOLOv11 + UKF + BoT-SORT". The frontend repository should be named "umpire-ai-frontend" with the description "Umpire AI frontend - React 18 + TypeScript video analysis dashboard". Do NOT initialize either repository with a README, .gitignore, or license, as these files already exist in the project.

8.2 Personal Access Token

To push code from the command line, you need a GitHub Personal Access Token (PAT). Navigate to github.com/settings/tokens, click "Generate new token (classic)", and select the "repo" scope for full repository access. Set the expiration to your preferred duration (90 days recommended), give it a descriptive note like "Umpire AI Deployment", and click Generate. Copy the token immediately as it will not be shown again. When prompted for credentials during git push, use your GitHub username as the username and the Personal Access Token as the password. The token can be revoked at any time from the same settings page.

8.3 Push Commands

For the backend repository, navigate to the umpire-ai-backend directory, initialize git, configure your user name and email, add all files, commit with a descriptive message, add the remote origin, and push to the main branch. The same process applies to the frontend repository. When pushing for the first time, Git will prompt for credentials - enter your GitHub username and Personal Access Token. After the initial push, Git will cache the credentials for subsequent pushes.

9. Complete File Structure

File / Directory	Component	Purpose
app/main.py	Backend	FastAPI app factory, CORS, lifespan hooks
app/config.py	Backend	Pydantic Settings (all configuration variables)
app/core/ball_detector.py	Backend	Hybrid YOLOv11 + HSV ball detection
app/core/ball_tracker.py	Backend	Hybrid BoT-SORT + Hungarian tracking
app/core/trajectory_estimator.py	Backend	UKF physics-based trajectory estimation
app/core/pitch_mapper.py	Backend	Perspective transform + auto-calibration
app/core/decision_engine.py	Backend	ICC rules: LBW, Bowled, Wide, No-ball
app/services/video_processor.py	Backend	Pipeline orchestration + video annotation
app/api/routes/upload.py	Backend	Video upload + metadata endpoints
app/api/routes/analysis.py	Backend	Analysis jobs + results + SSE stream
app/models/schemas.py	Backend	Pydantic request/response schemas
app/utils/drawing.py	Backend	OpenCV drawing helpers for annotations
app/utils/frame_utils.py	Backend	Frame extraction, encoding, video info

requirements.txt	Backend	Python dependencies (FastAPI, OpenCV, etc.)
Dockerfile	Backend	Multi-stage Python 3.11 Docker image
railway.toml	Backend	Railway deployment configuration
render.yaml	Backend	Render.com deployment configuration
src/App.tsx	Frontend	Root component with React Router
src/pages/UploadPage.tsx	Frontend	Video upload with drag-and-drop
src/pages/ProcessingPage.tsx	Frontend	6-step pipeline progress visualization
src/pages/AnalysisPage.tsx	Frontend	Frame-by-frame analysis dashboard
src/components/video/FrameByFramePlayer.tsx	Frontend	HTML5 video with frame stepping
src/components/video/VideoControls.tsx	Frontend	Speed, loop, keyboard controls
src/components/analysis/TrajectoryCanvas.tsx	Frontend	Canvas overlay trajectory rendering
src/components/analysis/PitchMap.tsx	Frontend	2D top-down pitch visualization
src/components/analysis/DecisionPanel.tsx	Frontend	OUT/NOT OUT decision display
src/components/analysis/StatsPanel.tsx	Frontend	Speed, deviation, confidence stats
src/hooks/useVideoUpload.ts	Frontend	Upload state management hook
src/hooks/useFramePlayer.ts	Frontend	Frame stepping control hook
src/hooks/useAnalysis.ts	Frontend	Analysis data fetching hook
src/lib/api.ts	Frontend	Centralized API client module
src/types/index.ts	Frontend	TypeScript interfaces for all API types
package.json	Frontend	Node.js dependencies and scripts
Dockerfile	Frontend	Multi-stage Node build + nginx serve
nginx.conf	Frontend	SPA routing + API proxy configuration
vercel.json	Frontend	Vercel deployment configuration

Table 7. Complete Project File Structure (38 files)

10. Troubleshooting

This section covers common issues and their solutions for both development and production environments.

10.1 CORS Errors

If the browser console shows CORS errors when the frontend calls the backend API, check that the backend's CORS middleware allows the frontend's origin. In development, the backend uses allow_origins=["*"] which permits all origins. In production, update app/main.py to specify your exact frontend domain. Alternatively, set the CORS_ORIGINS environment variable to a comma-separated list of allowed origins.

10.2 Connection Refused

If the frontend cannot reach the backend, verify that the VITE_API_URL environment variable is set correctly and points to the deployed backend URL. Test backend accessibility by visiting the /health endpoint in a browser or using curl. Check that the backend is running and that no firewall is blocking the connection. For Docker deployments, ensure the nginx reverse proxy is correctly routing /api/* requests.

10.3 Video Upload Failures

Large video uploads may fail with a 413 error if the nginx configuration does not allow sufficient body size. The provided nginx.conf sets client_max_body_size to 500M, which should be sufficient for most cricket match clips. If using a different nginx configuration, ensure this value is set appropriately. Also check that the backend's MAX_VIDEO_SIZE environment variable matches your needs.

10.4 Slow Processing

Video processing speed depends on the hardware and configuration. To speed up processing, increase the FRAME_SKIP parameter from 2 to 3 or 4 (processes fewer frames). Reduce YOLO_IMGSZ from 1280 to 640 for faster inference with slightly reduced accuracy. Use DETECTION_TIER=classical to skip YOLO entirely if no GPU is available, as HSV detection is faster on CPU. On Railway's free tier (512MB RAM), large videos may cause out-of-memory errors; upgrading to the hobby plan ($5/month, 1GB RAM) is recommended.

10.5 YOLO Model Not Found

If the YOLO model file is not found at the configured path, the system automatically falls back to HSV color segmentation. This is the expected behavior on CPU-only servers or fresh installations without a trained model. To use YOLO detection, train a model on cricket ball data using the provided training script, save the weights as yolo11s_cricket_ball.pt, and place the file in the project root directory. A pre-trained cricket ball dataset is available on Kaggle for fine-tuning.